XY Universe: a 2D particle environment for Deep Reinforcement Learning

Anson Wong*
Stadigi Labs, 1470 Peel Street, Suite 720 Tower B, Montreal, QC, H3A 1T1

We provide here a platform 2D particle environments for training agents in a continuous (x, y) observation space and discrete action space.

I. INTRODUCTION

With deep reinforcement learning
Self-play can produce complex behaviour in simple environments [**?**]

II. ENVIRONMENTS

A. Survive

B. Escape

C. Bump

III. AGENT MODELS

To demonstrate that our environments are useful and complex, we train an agent to beat the games. We try a set of agents:

Deep Q-Networks (DQN) are neural networks that approximate the state-action value $Q(s, a)$ which be definition is

$$Q^{\pi}(s,a) = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty}\gamma^{k} r_{t+k} | s_t = s, a_t = a\right] \quad (1)$$

$$= \mathbb{E}_{\pi}\left[r_t + \gamma r_{t+1} + ... | s_t = s, a_t = a\right] \quad (2)$$

For the optimal Q-value

$$Q(s,a) = \mathbb{E}\left[R(s \to s', a) + \gamma \max_{a'} Q^{*}(s', a') | s, a\right] \quad (3)$$

The optimality condition is

$$Q(s,a) = Q \quad (4)$$

The update rule is:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\left[Q_{\text{target}}(s_{t+1}, a_{t+1}) - Q(s_t, a_t)\right] \quad (5)$$

where

$$Q_{\text{target}}(s_{t+1}, a_{t+1}) = r_t + \gamma Q(s_{t+1}, a_{t+1}) \quad (6)$$

In the framework of deep learning our goal is to minimize the difference between Q and Q_{target} through an L2 loss function of

$$L_t = \left(Q_{\text{target}} - Q(s_t, a_t)\right)^2 \quad (7)$$

Note that this loss function

IV. TRAINING

In practise, approximating Q using an update equation 5 after every state transition poses a deep learning problem that is different than the typical supervised deep learning problems because our training data target values are not fixed as seen by the t-dependence in Eqn 7 because of recursive relationship in the Bellman equation. This difference gives rise to harder to train because the target constantly moves. There are many heuristic ways to help with the training using neural network back propagation, we will consider here Prioritized Experience Replay and Target Network Freezing.

Prioritized Experience Replay where we train our agent network from randomly sampling a subset from a pool of past experiences $\{s_i, a_i, R_i, s_{i+1}\}_{i=1}^{M}$, where M is the size of the memory pool, to help reduce the heavy recency bias of experiences in the training data. Specifically in our implementation, we leave the training for after the generation of episodes where we randomly batch train N times on our experience memory pool.

Target Network Freezing involves cloning our model network's weights for producing Q_{target} values for a period of iterations before updating them by re-cloning our most recent model weights. The heuristics of introducing a target network is to help with reducing the training chase of a moving target, with the trade-off of possibly countering labels.

V. ENVIRONMENT PHYSICS

- particle-wall elastic collisions
- particle-particle elastic collisions

* e-mail: anson.ao@gmail.com
github: github.com/ankonzoid

VI. AGENT OBSERVATION SPACE

In simple reinforcement learning problems with a low state dimensions, one usually solve the problem set the observation space of the agent to be.

As we increase the state dimensions of environment, we tend to run into a few problems if we maintain setting the observation space of the agent as the entire state space:

- The state space might be too large to physically explore all state and action combinations
- We might miss out imposing useful and important physical symmetries, that will instead have to be exhaustively searched for the agent and making training a much harder job

The field which deals with setting the observation space to be a subset of the state space is called partially-observable Markov decision processes (POMDP) where there are hidden states that are beyond the what the agent can observe.

VII. RESULTS

Although watching videos is likely the most convincing method of confirming an agent has been trained to beat games, one can also introduce related environment-dependent metrics to evaluate agent performance.

VIII. SUMMARY